GenAI 赋能工业编程

基于 Agent 的 PLC ST 代码生成与逐周期验证

课程期末大作业实验报告

团队	五人小组
版本	v1.0
交付日期	2026-11-09
实验口径	34 题 × 4 组 = 136 run

本报告区分正式实验、工具验收与 fixture 演示

目录

摘要 02
第 1 章　实验环境与复现边界 03
1.1 软件与模型 03
1.2 编译工具链 03
1.3 模型与基线 03
1.4 复现流程 04
1.5 证据边界 04
第 2 章　数据集与形式化规约 05
2.1　数据集构成 05
2.2　来源与改编流程 05
2.3　数据需求的合理性 06
2.4　多样性的衡量方式 07
2.5　形式化规约 07
2.6　校验结果 09
2.7　本章小结 09
第 3 章 自研 Agent 框架设计与实现 10
3.1 总体架构及五项能力 10
3.2 数据适配、任务卡与声明骨架 11
3.3 分层上下文与模型服务 12
3.4 多候选、双通道编译与功能判定 12
3.5 有界修复与回归保护 13
3.6 现成框架基线与公平性 13
3.7 日志、断点续跑与失败归因 13
3.8 验证范围、正式结果与已知限制 14
3.9 编译环境工程验收（不是模型对比结果） 14
第 4 章 实验结果与分析（统稿版） 16
4.1 实验设计与数据合并 16
4.2 指标口径 16
4.3 四组结果 16
4.4 难度分层与配对比较 17
4.5 失败归因与判据限制 18
4.6 资源消耗与可复现性 19
4.7 交付与后续处理 20
结论与局限 21
参考文献 22

摘要

本作业研究大语言模型能否在工业控制系统约束下生成可编译、可逐周期验证的 PLC Structured Text 程序。团队构建了 34 道题的数据集，其中 20 道来自课程公共题库，14 道为改编或自拟题；每题具有统一 YAML 字段、I/O 表、初始状态、编号约束和 LTL 规约，共形成 316 个 I/O、144 条约束和 177 条形式化条款。测试层包含 214 条正常、边界、异常和安全用例，以 20 ms 扫描周期逐拍比对，并使用缺陷单、变异测试和可选形式验证跟踪失败。

实验采用 2 x 2 设置：Aider 与自研 Agent 分别搭配 `deepseek-v4.1-flash` 和 `gpt-6.1-sol`，四组均覆盖全部 34 题，最终矩阵为 136 个 run。严格整题通过数分别为 E1 18/34、E2 16/34、E3 8/34、E4 7/34。结果说明，在这套题目、本机工具链和本次配置下，编译通过仍明显高于逐周期功能通过，Aider 组的严格整题通过数更高。该差异不能直接解释为算法普遍优劣：E3 有 24 题受预算终止影响，自研框架还存在字节长度误当 token 的预算实现问题；P19、S11、S05-06 的判据争议也尚未全部消除。

本作业的可复核贡献包括统一题集与形式化条款、具备生成与反馈闭环的 Agent、逐周期测试和缺陷追踪、四组可回指原始记录的实验，以及 Word/PDF、PPT 和 A1-A7 提交包。已知限制是换机完整复现尚未完成，精简实验包不包含全部逐拍 actual 中间文件，单次运行也不足以估计模型随机波动。结论只适用于当前冻结版本和实验条件。

关键词：工业编程；PLC；Structured Text；大语言模型；Agent；形式化规约；逐周期验证

第 1 章　实验环境与复现边界

本章登记正式实验实际使用的软件、模型和工具链。复现的目标不是让另一台电脑无条件产生完全相同的模型文本，而是让读者能判断当时用了什么、哪些步骤可以重算、哪些中间证据只存在于实验机，以及改变环境后哪些结论必须重新验证。

1.1 软件与模型

正式实验使用 CODESYS x64 3.5.22.40（V3.5 SP22 Patch 4）作为权威编译环境，目标设备为 CODESYS Control Win V3 x64。模板工程包含 Application、MainTask 和 20 ms 任务，核心库为 Standard 3.5.22.0 与 IoStandard 3.5.22.0。MATIEC 使用固定官方提交构建，承担快速语法检查；它不替代 CODESYS 的最终编译状态。

表 1-1 环境与版本

项目	版本或配置
操作系统	Windows 64 位，10.0.26200
Python	Agent 核心支持 3.10+；编译验收记录为 3.12.14
CODESYS	3.5.22.40，V3.5 SP22 Patch 4
CODESYS Scripting	4.2.0.0，IronPython 2.7
目标设备	CODESYS Control Win V3 x64，type 4096，id 0000 0004
核心库	Standard 3.5.22.0、IoStandard 3.5.22.0
扫描周期	20 ms
基线	Aider 0.86.2
模型 A	`deepseek-v4.1-flash`
模型 B	`gpt-6.1-sol`
模型接口	DMXAPI Chat Completions
单题超时	900 s
权威编译超时	90 s

硬件型号、CPU、内存和磁盘信息未在成员交接材料中留下可核对记录，因此本文不补写。正式实验在同一实验机上完成。换机复现需要重新记录硬件和软件，不能把本表直接视为已完成跨机验证。

1.2 编译工具链

编译器接入分两层。快通道调用 MATIEC，目标是在 3 s 内给出语法和类型诊断；权威通道复制专用 CODESYS 模板，导入 TYPE 和单个 POU，强制链接、重新构建并读取结构化消息。每个诊断统一为 `code / severity / line / message / raw`，警告与错误分开保存。

CODESYS 验收包括 6 个诊断探针和 34 个声明骨架。正确例、常量死分支和 4 个故意错误例均符合预期；34 个骨架中 33 个通过并生成目标代码，P18 因当前 profile 不支持 `VARIANT` 失败。该结果只证明接口与方言兼容性，不属于模型生成程序的编译通过率。

1.3 模型与基线

模型调用通过 DMXAPI 的 Chat Completions 格式完成。Key 从 `DMXAPI_API_KEY` 环境变量读取，不写入配置或日志。两个模型 ID 在正式矩阵中固定，不在不同组别间替换。Aider 0.86.2 作为现成 CLI

Agent 基线，经 OpenAI-compatible 路由接入；它保留自身的上下文、编辑和反思策略，自研框架保留候选、局部补丁和回归保护策略。

公平比较依靠共同输入和共同判定：两框架使用同一 34 题、同一 I/O 与 LTL、同一 CODESYS 权威编译、同一成员 4 逐周期测试器、同一 900 s 外部超时和同一 22 字段日志契约。两种框架内部策略无法完全相同，这属于研究对象的一部分，也是结论必须限制在“本版配置”上的原因。

1.4 复现流程

无费用的最小验收包含三步：安装依赖、运行单题 fixture、运行 34 题 fixture。三条命令检查解析、上下文、候选、修复、终止和日志流程，不调用模型或 CODESYS，不能计入实验通过率。

真实重跑需要另外准备模型 Key、Aider 独立环境、CODESYS profile、模板工程和测试执行器。换机后必须先做声明骨架和诊断探针，再跑少量含失败样本，最后才可按四组配置生成新批次。新批次应使用新的目录和 run_id，不能覆盖本作业的 136 条最终记录。

1.5 证据边界

正式实验交付 136 个最终 run、32 个判据修订补跑和 104 个沿用首轮记录。每个 run 保留 result、metadata、候选评估、事件摘要和最终 ST；完整逐拍 actual 与编译中间工程只保存在实验机，精简包中没有。因此本仓库能重算汇总、定位到 run 和具体失配记录，但不能仅靠提交包重新执行全部 CODESYS 周期。

这一边界直接影响结论：编译与逐周期测试的结果可核验，模型服务端的状态和 complete actual 不能在本仓库内独立复现。报告中的 token、时延和通过数必须与这些限制一起阅读。

第 2 章　数据集与形式化规约

本章负责回答课程要求的三条数据类问题：数据集从哪来、改编记录是否可追溯、多样性怎么衡量；以及一条规约类问题：公共题与自编题是否都写了合理且覆盖较全面的 LTL。对应产物为 `data/` 整包（题面、规约、来源表、多样性矩阵、评审记录）。

2.1　数据集构成

数据集由两类题组成：课程提供的 **20 道公开题**（P01—P20）与本组自编的 **14 道题**（S01—S14），合计 **34 道**。全部题面统一为 12 字段 YAML，全部规约统一为带条款编号的 LTL 文本，二者按题号一一对应。规模如下：

统计项	数值
题目总数	34（公开 20 + 自编 14）
I/O 点合计	316
约束（K）合计	144 条
LTL 条款合计	177 条（平均 5.2 条/题）
难度分布	易 9、中 14、难 11
类别分布	A 6、B 11、C 11、D 6

公开题采用**原文导入**而不是手工重写：题库已经给出结构化的需求正文与接口定义表（变量名、方向、数据类型、说明），手工再抄一遍只会引入错字。导入脚本 `tools/import_bank.py` 负责解析与校验，人只做三件事——核对 I/O 与需求是否自洽、补上题库没给的初始状态与错误示例、为每题选条款。省下来的时间全部投到自编题与规约上，这两样才是评分点。

2.2　来源与改编流程

2.2.1　四类来源

来源类型	题数	说明
课程公开题库（原文导入）	20	需求与 I/O 表逐字保留，仅补初始状态、约束编号、错误示例
改编自公开题	8	保留原题的场景骨架与控制难点，改工艺对象、改 I/O 规模、加一条原题没有的约束
自拟	6	来源为现场工艺惯例（CIP 清洗流程、NAMUR NE43 传感器故障指示、液压分级加载）或课程示例的分级停机思路

2.2.2　生成与改编流程

自编题按六步生成，顺序不能颠倒——先想「什么样的实现算错」，再写需求：

1. **选场景**：从真实 PLC 工位里挑一个能在一页纸内说清、且有明确对错判据的动作序列；说不清对错的场景直接放弃。

2. **写错误示例**：先写 1—2 条「典型错误实现 + 症状」（例：忘记互锁导致双泵同时启动）。写不出错误示例的题说明它没有判据，换掉。

3. **反推约束**：把错误示例逐条转成编号约束 K1…Kn，每条点名具体信号与时间量。

4. **定 I/O**：控制在 6—12 个点；超过 12 个就拆题（P19 的 24 点属于公开题，自编题不做这么大）。

5. **写规约**：从条款库 C1—C10 中按题型选 4—6 条，逐条标注验收口径与对应 K。

6. **交叉评审**：由成员 4 站在用例作者角度复核，回答「我能不能只凭这份题面写出逐周期用例」；不能就改题面，不改用例。

2.2.3 来源表字段（data/provenance.csv，34 行 × 11 列）

列	含义	举例
task_id	题号	S07
source_type	课程公开题库 / 改编自公开题 / 自拟	改编自公开题
source_link	可点开的原始出处（标准号、官网、赛题页）；课程题库无外链时填下发渠道	https://www.siemenscup-cimc.org.cn/project/index
source_ref	精确到条目或条款的出处，让人能按图索骥	大作业题库.json：easy[4] / 题号 05；ISO 4413:2010 §5
original_title	原题标题；自拟题填破折号	Automated Filling Line Control
synthesis_process	这道题是怎么合成的（导入步骤或六步推导）	按六步流程合成：选场景 → 先写错误示例 → 反推约束 K → …
adaptation_what	改了什么（可核对的具体改动）	四工位改单工位、去掉包装段、灌装加超时 5 s 错误码
adaptation_why	为什么这样改（要说明它带来了什么新的判定点）	原题不考节拍超时，加上才能测出「卡步」实现
author / reviewer / review_date	出题人、复核人、复核日期	成员 1 / 成员 4 / 2026-10-30

2.2.4 原创度

课程要求自建题库「独立创建」，因此改编题与自拟题还要说清与来源题的关系。本组的分类口径是：**改编**＝借原题的场景骨架与控制难点，接口可以整个换掉；**自拟**＝连场景也自己找（现场工艺惯例或标准）。判据用三个量：变量名重合率（有没有照抄接口）、接口规模（点数是扩是缩）、类型构成 Jaccard（接口形态像不像）。完整逐题表在 `data/diversity.md` 第 6 维。

需要说明这张表**不能证明什么**：公开题沿用英文原名、自编题按中文工艺对象重新命名，名字天然难重合，所以低重合率只说明「没有照抄接口」，不说明「与来源题无关」。改编幅度真正的证据是 `adaptation_what` 列与题面的 `error_examples`——后者写明这道题新考了什么、哪一种错误实现会被它抓住。

全 14 道自编题中，**6 道完全自拟**（只借用现场工艺惯例，接口全部新设），其余 8 道为改编；改编题的改动都落在工艺对象与接口规模上，因此多数与来源题的 I/O 重合率低于 50%。来源类型、改动内容与理由三列在 `data/provenance.csv` 里逐行可查。

2.3 数据需求的合理性

「结合实际 PLC 场景」不是一句声明，本章用三条可检查的标准来支撑：

第一，**每道题的输出都对应真实执行器**。清洗泵、灌装阀、变频器、分拣气缸、阀门与泵组，不是抽象的变量名；错误动作在现场都有物理后果，因此「安全侧默认值」（停机、关阀、不输出）在这些题里是可写的、也是必须写的。

第二，**时间量取自现场常识而不是随手编**。去抖 500 ms、启动延时 5 s、超温确认 3 s、CIP 三步 30/60/30 s、班次统计按 LTIME 累计——这些数值都能在工艺文件里找到对应说法，并且都被换算成扫描周期写进了规约（20 ms 为一周期）。

第三，**错误处理按工业惯例分级**。以 S11 为例：4—20 mA 回路用 3.6/21 mA 判断断线与超量程、用 3.8/20.5 mA 判断恢复，这是 NAMUR NE43 的通用做法；降级运行时输出切到备用值而不是停在失效测量值上，这对应现场「仪表坏了不能带着坏值继续跑」的硬要求。再如 S09 的 CIP：三路阀门互斥是防止清洗液与清水串流，泵必须先开阀再启动，这些约束在现场都是安全规程而不是编程偏好。

2.4 多样性的衡量方式

多样性不靠题目数量堆，而是把 34 道题投影到六个正交维度上逐格检查（完整矩阵见 `data/diversity.md`，由脚本现算）：

维度	取值	结论
场景类别 × 难度	A/B/C/D × 易/中/难	12 格中 11 格有题；空格为设计选择（见下）
I/O 数据类型	BOOL/INT/DINT/UINT/UDINT/REAL/WORD/TIME/LTIME/ARRAY	10 类全部覆盖，单题 2—24 点
程序结构	纯组合/带定时/带数组/带状态机/带累计	五类均有题，一题可带多个标签
验收重点	C1—C10 十个条款类型	十类全部有题覆盖，无「只压一种条款」的题
来源结构	原文导入/改编/自拟	20 / 8 / 6
原创度	自主设计/接口重构/大幅改编/局部改编	自拟 6 道接口全新；改编题 I/O 重合率逐题可算

两处空格及其原因：**A 类（开关量联锁）没有难题**——联锁一加复杂度（节拍、状态机、故障分级）就自然落到 C 或 D 类，硬在 A 类里堆难题只会造出伪难题；**D 类只有 1 道难题**，安全类题目一旦要求复杂的恢复时序，本质上已经是状态机题。这两处写在 `diversity.md` 第 6 节，属于主动声明的短板，不是遗漏。

2.5 形式化规约

2.5.1 为什么先建条款库

上一版方案要求「每题 6 条 LTL」，实践下来必然凑数：第 6 条往往是前 5 条换一种写法，看起来覆盖全面，实际上判不出新东西。本版改为**先建十条的条款库 C1—C10，再按题型选 4—6 条**，并要求逐题标注哪些条款不适用、为什么。覆盖是否全面由映射表证明，而不是由条数证明。

条款	管什么	典型形式
C1 停止优先	安全	`G(!EStop_NC -> !Motor)`
C2 启动条件	安全	`G(Motor -> (EStop_NC && SensorOK))`
C3 自保持	保持	`G(Valve -> (Valve W Done))`
C4 输出互斥	互斥	`G(!(Fwd && Rev))`
C5 响应时限	时序	`G(Fault -> F0,25)`
C6 报警锁存	保持	`G(Alarm -> (Alarm W !Cause))`
C7 滞回/不抖动	时序	`G(HiAlarm -> (HiAlarm W Value < hi - hyst))`
C8 计数单调	保持	`G(Overflow -> X(Overflow))`
C9 定时下界	时序	`G(Step -> G0,5)`

条款	管什么	典型形式
C10 初态与复位	恢复	`G0,1`

2.5.2 写法约定

- **原子命题与 I/O 同名**，不另造变量；数组元素用 `# alias:` 声明别名（P08 的 `chEn/chOut/chRet`），数组的逐元素语义交给用例。
- **时间区间以扫描周期为单位**：`[0,25]` 即 500 ms。文件头固定写 `# spec: … scan cycle 20 ms`。
- **每条条款一行**，格式 `C<n>: <公式>  -- <验收口径>`（K 编号），条款号在同一题内唯一，同类条款重复时加后缀（C2b、C7b）。
- **可解析**：`tools/ltl_check.py` 是自写的递归下降解析器，规约必须能被它读通——「人能读但工具读不了」的规约不算规约。

2.5.3 逐题条款概览

题号	条款数	用到的条款	覆盖的 K
P01	6	C1、C2、C3、C4、C6、C9	3/4
P02	5	C2、C7、C8、C10	3/4
P03	6	C1、C2、C4、C7、C10	4/4
P04	5	C1、C2、C7、C8、C10	4/5
P05	5	C1、C2、C6、C9、C10	4/4
P06	5	C1、C2、C3、C6、C10	3/4
P07	5	C1、C2、C4、C6、C10	2/4
P08	5	C1、C2、C6、C10	2/5
P09	5	C1、C3、C2、C10	2/4
P10	6	C1、C2、C7、C9、C10	3/4
P11	5	C4、C2、C3、C6、C10	3/4
P12	5	C2、C5、C6、C8、C10	2/4
P13	5	C2、C3、C6、C4、C10	3/4
P14	2	C2、C10	1/4
P15	5	C1、C2、C4、C6、C10	2/4
P16	5	C4、C2、C1、C9、C10	2/4
P17	5	C1、C2、C4、C6、C10	4/4
P18	5	C1、C4、C2、C9、C10	3/4
P19	6	C1、C2、C4、C7、C6、C10	3/5
P20	5	C4、C2、C1、C6、C10	2/4
S01	5	C1、C2、C4、C10	3/4
S02	6	C1、C2、C4、C9、C10	4/4
S03	6	C1、C5、C2、C6、C10	4/4
S04	5	C4、C2、C6、C7、C10	3/4
S05	6	C1、C2、C8、C6、C10	3/4
S06	6	C1、C2、C4、C7、C8、C10	3/4
S07	6	C1、C2、C4、C6、C9、C10	4/5
S08	5	C1、C2、C4、C9、C10	3/4
S09	5	C1、C2、C4、C5、C10	4/5
S10	5	C1、C2、C6、C7、C10	4/4
S11	4	C1、C2、C6、C10	3/5
S12	5	C1、C2、C3、C4、C10	3/4
S13	6	C1、C4、C5、C6、C7、C10	5/5
S14	6	C1、C2、C4、C6、C7、C10	5/5

2.6 校验结果

数据与规约都由脚本自检，不靠人眼：

脚本	检查什么	结果
`tools/ltl_check.py`	LTL 语法、命题能否溯源到本题 I/O 或别名、条款号范围、条款数 4—6	34 份规约、177 条条款全部通过
`tools/schema_check.py`	12 字段、I/O 方向与类型、K 编号映射、来源表覆盖、难度与类别统计	通过；38 条约束仅由用例覆盖（警告）
`tools/gen_meta.py`	来源表、多样性矩阵、冻结哈希是否与数据一致	34 行来源表、6 维矩阵、68 个文件哈希

评审方面，34 道题全部经成员 4 二次复核，共发现 6 类问题并闭环（其中 3 条是规约与题面矛盾，会直接导致下游返工），完整记录见 `data/review.md`。

2.7 本章小结

本章交付的是一份**可判定的题集**：题干、I/O、初始状态、编号约束、错误示例五样齐全，每题配 4—6 条能被工具解析的 LTL 条款，来源可追溯到题库条目或工艺惯例，多样性由 6 维矩阵逐格证明，短板主动声明。它直接支撑后面三件事：成员 2 的框架有了稳定的输入格式，成员 4 的用例有了判据（条款号 + K 编号），成员 3 的报告有了可引用的数据规模与口径。

第 3 章 自研 Agent 框架设计与实现

本章说明成员 2 交付的框架结构、统一工具接口、候选与修复策略，以及结果可追溯机制。实现对象是由自然语言需求、I/O 表和 LTL 规约生成 PLC Structured Text 源码的课程实验。自研框架负责组织和执行反馈闭环；PLC 功能正确性仍由独立的编译、逐周期测试与可选形式验证判定。

本次交付包含可运行的框架源码、Aider 接入层、配置与提示词、离线演示和接口文档。本机已安装 CODESYS 3.5.22.40 与 MATIEC 并完成编译环境验收；成员 4 的逐周期执行器和用例已接入正式实验。模型 ID 固定为 `deepseek-v4.1-flash` 与 `gpt-6.1-sol`，Aider 版本为 0.86.2。四组最终实验共 136 个 run，结果见第 4 章。fixture 仍只验证框架控制流程，不进入正式统计。

3.1 总体架构及五项能力

框架使用 Python 实现。任务书指定的七模块保留在 `agent/src/`，统一入口为 `agent/run.py`。自研模型调用和现成 Aider 基线通过同一入口调度，最终都使用同一套编译、功能测试和结果记录方式。架构图与执行流程图见 `docs/framework.md`，可直接用于报告配图。

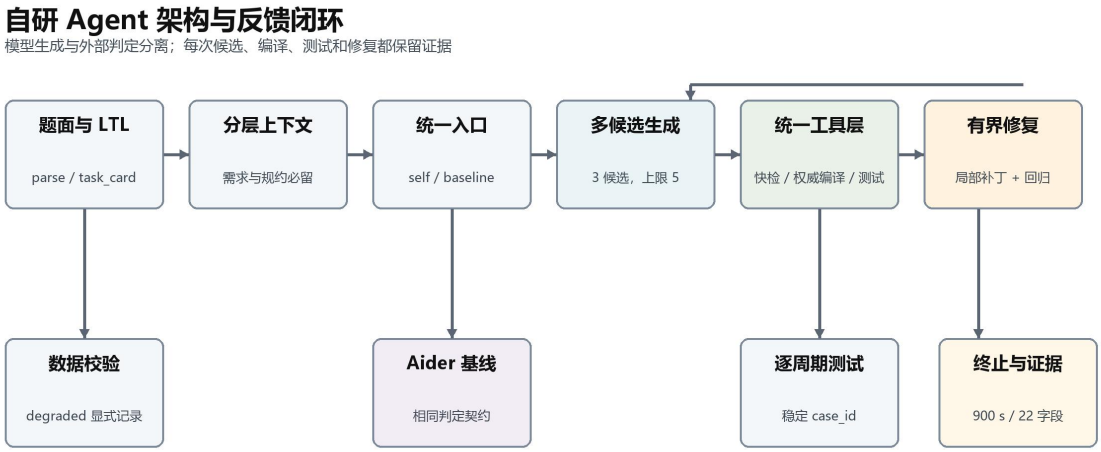


图 3-1 自研 Agent 架构与反馈闭环

表 3-1 模块、责任及产物

模块	文件	输入	主要输出
解析	parse.py	YAML、LTL	task_card.json、缺字段与语义告警
任务卡与上下文	task_card.py	任务卡、参考、本轮诊断	声明骨架、分层上下文、裁剪 trace
生成	generate.py	上下文、模型配置	多候选 ST、API 响应缓存与 usage
工具层	tools.py	ST 源码、工具命令配置	双通道诊断、用例结果、验证结果
修复	repair.py	诊断、源码、失败用例	局部补丁、差异、回归判定
终止	terminate.py	候选状态、时间、预算	选择排序与终止原因
日志	logging.py	结果与过程事件	22 字段终态、JSONL 事件、原子写入文件

五项能力具有明确的代码落点。任务组织由解析和批量入口完成，题号、设置、重复次数和 seed 确定 run_id；上下文管理由任务卡模块完成，确保需求与规约不因预算不足被悄悄截断；工具调用由工具层封装；编译与修复由入口和修复模块构成闭环；终止由轮数、停滞、预算及父进程硬超时共同约束。此划分把模型生成与外部判定分开，便于成员 4 替换测试执行器，也便于成员 3 在相同判据下比较框架。



图 3-2 单题执行与终止流程

3.2 数据适配、任务卡与声明骨架

上游冻结数据为 v1.7，含 34 题，即 P01-P20 和 S01-S14，共 144 条约束、177 条 LTL 条款。任务书原计划 32 题，但当前最终矩阵统一采用 all34，四组主轮为 136 跑。批量入口仍保留 core32 作兼容选项，但任何正式结果都必须声明题集。采用 all34 后，32 个判据修订补跑作为替换记录进入最终矩阵，不与首轮数量相加。

解析器读取任务书 10 个必填字段，保留 error_examples 和 notes 等可选字段；题面原始字节 SHA256 前 8 位作为 task_version，规约另记完整哈希。缺字段、缺规约或无效 I/O 会显式标记 degraded，默认 live 模式不继续该题，不猜补需求。成员 1 校验器已检查 34 题 schema 与 LTL 语法通过，但这只证明数据结构和规约语法/映射，不证明语义无冲突。

对复杂接口，任务卡保留原始 io_table 并另建可声明接口投影。例如结构成员依据原题显式字段组成 TYPE；channels[].成员映射为父数组元素 STRUCT 成员。开放数组、VARIANT 和 LTIME 保留原类型并写入 dialect_requirements，不将它们替换成“容易编译”的固定类型。骨架使用 FUNCTION_BLOCK、VAR_INPUT、VAR_OUTPUT 和 VAR_IN_OUT 声明。自然语言初态交给生成阶段实现，解析阶段不臆测赋值或定时器状态。

目前解析器对 P08/P14/P17/P19 的需求、约束、接口或规约矛盾给出 semantic_warnings。例如急停极性、错误状态码和缺少返回状态接口会影响验收，不能由模型自行选择。允许降级试跑时告警仍保

留，metadata 不将该样本自动判为可用正式实验数据。应由成员 1 和成员 4 先统一判据、更新冻结记录，再重跑。所有声明骨架的 skeleton_compiled 初值为 false，真实编译器运行后才能给出编译证据。

3.3 分层上下文与模型服务

上下文按题面、规约、同类参考和本轮错误四层组织。需求原文、I/O、初态、约束、数据告警和完整 LTL 始终保留；参考最先裁剪，仍超限时可删除本轮反馈，且记录删除原因。若必留内容自身超过预算，流程以 budget 终止，不在后台截断任务。不同轮次重新组装反馈，不累积旧错误。裁剪决定保存在 context_trace.json，便于解释每段材料为何保留或删除。

上下文预算预设 6000 token。未绑定具体模型分词器时，离线估算使用 ASCII 约 3 字符/token、非 ASCII 每字符 2 token，并为固定提示保留余量；这属于本地估计，不是远端模型精确计数。total_token_budget 还为累计请求设置本地预算门槛。真实成本统计只使用服务商返回的 usage；usage 缺失时 tokens_in 或 tokens_out 记 null，不把估算值冒充账单。正式实验须记录模型上下文上限与准确计费方式。

自研模型服务适配 DMXAPI Chat Completions 接口。base URL 为 `https://www.dmxapi.cn/v1`，请求追加`/chat/completions`，使用标准库 HTTP 发送 model/messages/stream 等字段。Key 只从环境变量读取，不进入配置或日志。正式实验使用 `deepseek-v4.1-flash` 和 `gpt-6.1-sol` 两个准确模型 ID。

候选策略包括直接实现、边界与联锁优先、初态与状态保持优先。温度列表预设 0.2/0.6/0.9，但不同模型对参数支持不同，默认配置不发送 temperature 与 seed；启用后必须冻结并在实验设置中说明。相同 seed 只是复现记录的一部分，服务端模型更新、网关调度及采样仍可能引入差异，不能保证完全相同源码或结论。

3.4 多候选、双通道编译与功能判定

默认每题生成 3 候选，上限 5，可配置为 1 以验证单候选行为。先对候选执行编译过滤，再按通过用例数、修复轮数、token 消耗排序。排序实现可独立测试，避免依赖人工阅读代码“挑看起来更好的一份”。生成候选数和实际修复轮数进入终态日志。

快通道面向 MATIEC 或 RuSTy，预设 3 秒；权威通道面向 CODESYS 脚本编译，预设 90 秒。所有诊断归一化为 code、severity、line、message、raw。warning 与 error 分开保留，死代码警告不能自动判成编译失败。正式编译通过必须有权威工具的结构化结果及原始输出；未配置命令、进程异常或结果协议错误均记工具故障，不返回假 pass。具体脚本、命令占位符与 profile 接通步骤见 `docs/toolchain.md`。

编译通过后调用成员 4 功能执行器。功能桥返回总用例数、通过数、稳定的 passed_ids 及带 case_id/周期/输入/期望/实际的失败摘要。没有用例时记 skipped，不能把 0/0 记为全部通过。测试失败只说明被当前用例检出，当前用例全部通过也不证明对所有输入均正确。可选验证通过独立桥执行，未配置则 skipped；PLCVerif 类验证结果由成员 4 负责提供。

3.5 有界修复与回归保护

修复将错误分为语法、类型、未声明变量、缺库和逻辑失配，选择对应提示模板。反馈保留标准化诊断；有行号时仅附错误位置±3 行代码，无行号时不把全部实现灌回模型。逻辑失败回灌成员 4 提供的用例摘要。模型须返回 JSON 增量替换列表，而不是整文件重写。

补丁中每个 old 必须非空且在当前源码中唯一出现，new 必须为文本；单轮修改处数和替换跨度受限制。不符合协议的补丁记录为拒绝，不直接覆盖文件。每轮保存差异与评估，以回答“第几轮改了什么”。修改后重新编译并运行全部用例；已编译通过的程序退化为编译失败，或者已通过 case_id 集合减少，则回滚旧候选。该保护依赖测试执行器正确、稳定地报告用例 ID，不能替代真实回归实验。

修复进展另按编译是否通过、通过用例数、诊断错误数量比较；这样错误减少但尚未完全编译通过的中间补丁可以被保留，不因候选初选中的轮次惩罚被反复丢弃。最多修复 5 轮，连续 2 轮没有改善提前停止。预算与单题时间在每个阶段检查。外层父进程还为整题设置 900 秒硬上限，超时终止工作进程树并保留最近候选、编译输出和日志，回收批量槽位，继续后续题。900 秒是单题上限，不保证整批 8 小时完成；32 题口径 176 跑、并发 2 的全超时理论上界约 22 小时，34 题 184 跑约 23 小时，实际时长须测量。

3.6 现成框架基线与公平性

基线选用任务书允许的等价开源 CLI Agent Aider。其官方接口支持 OpenAI compatible base URL 和 openai/模型前缀，也支持 message-file 非交互执行以及外部测试命令反馈。Aider 接口说明、脚本执行说明、编译与测试接入。

基线每题在独立目录编辑 ST，通过`--auto-test --test-cmd` 调用 `baseline/judge.py`，在 Aider 每次编辑后使用同一 Toolchain 执行编译和功能测试；非零退出及诊断由 Aider 自己的反思策略处理。外部统一入口再使用相同判定及结果 schema 记录终态。基线不再套用自研多候选和局部补丁策略，否则比较对象会混合。各框架自己的上下文和修复策略允许不同，但题集、基础需求、可用工具、超时与最终判据必须相同，策略差异必须在环境表记录。

查阅时 Aider 官方参数并未定义`--max-reflections`；原生反思上限属于所装版本内部策略，不能凭空添加“五轮反思”CLI 参数。正式实验使用 Aider 0.86.2，并保留其原生行为记录。Aider 参数源码、反思控制实现。若基线使用原生 3 次反思而自研允许 5 轮，应在比较局限中明确说明；同一 900 秒外部上限不能消除所有内部预算差异。

3.7 日志、断点续跑与失败归因

单题结果严格保留任务书表 4-4 的 22 字段：run_id、setting_id、framework、model、task_id、task_version、repeat_idx、seed、start_ts、latency_s、candidates、repair_rounds、compile_status、compile_stage、compile_errors、test_total、test_passed、verify_status、final_status、artifact_path、tokens_in、tokens_out。过程事件、首轮编译状态、工具故障细分和实验资格存放在旁路 metadata 及 events，不往结果行随意加字段。

run_id 使用设置/题号/重复次数/seed。每题 manifest 指纹覆盖模型、参数、题面、规约、源码和用例摘要；已有终态且指纹一致时跳过。同 run_id 数据或配置变化则拒绝覆盖，要求新输出目录。批量汇总从每题 result.json 重建，不重复追加。指纹并未自动包含外部编译器二进制、库和远端模型真正版本，因此仍需人工登记环境。

模型请求按内容哈希缓存。发送前写 pending 标记；收到并成功保存响应后清除 pending。网络超时或进程崩溃造成请求状态不明时，默认停止，人工核实服务商记录后才考虑显式重试，避免自动重复计费。独占 running.lock 防止同题并发写；崩溃遗留锁必须先核实其中 PID 及子进程均已退出，再移除该题锁。已完成结果可续跑并不意味着所有中断场景都零额外费用。

final_status 提供 passed、best-effort 与 abandoned 及原因。模型编译/功能失败、工具故障、数据降级、超时和预算耗尽分开登记。compile_status 沿用任务书三态，计算正式通过率时必须结合 metadata 排除 fixture、数据降级及工具故障，不能把所有 fail 直接归咎模型。超时保留独立状态，不与普通模型失败混算。

3.8 验证范围、正式结果与已知限制

本包的自动检查覆盖解析与上下文裁剪、候选排序、补丁协议、回归回滚、标准化诊断、超时、日志字段及断点续跑等框架行为；具体测试数和执行结果以交付包实际检查记录为准。可控 fixture 演示覆盖成功、最佳努力、放弃、超时和工具错误路径，但不执行真实 PLC 语义。

正式实验使用四组各 34 题，最终严格整题通过为 E1 18、E2 16、E3 8、E4 7。该结果描述冻结题集、同一实验机和本版框架的一次运行，不证明 Aider 算法普遍优于自研框架。E3 的 24 次预算终止包含 UTF-8 字节长度误当 token 的已知实现问题，P19、S11、S05-06 仍有判据争议。完整逐拍 actual 中间文件只保留在实验机；精简包中的结果可回指到 run、候选评估和最终 ST，但不能在本包内重放全部周期。

换机复现模型、CODESYS 与 Aider 尚未完成。提交包提供环境、工具链和三条无费用命令；真实重跑必须重新登记版本并生成新批次，不能覆盖本报告使用的 136 条最终记录。

3.9 编译环境工程验收（不是模型对比结果）

CODESYS x64 3.5.22.40、GenAI 实例、Control Win V3 x64 目标、Standard 与 IoStandard 3.5.22.0、20ms 模板任务已完成本机验收。MATIEC 使用官方固定 commit 3a41303bcb4e50403417bc39c63f88981d7d961e 本机构建。

最终一次 IDE 启动完成 6 诊断探针和 34 声明骨架，约 121.4 秒。正确例和常量死分支通过，4 个故意错误例被拒绝；34 骨架 33 通过并生成目标代码，P18 因 VARIANT 未知类型失败。这衡量接口与方言兼容性，不是模型 CPR/EPR。完整双通道正确例约 33.8 秒（并发 2）、错误例约 32.2 秒（并发 2），均低于权威 90 秒上限；原生诊断已校准对象及 Decl/Impl 行号，无法可靠映射时仍为 null。

MATIEC 对 34 骨架 27 通过、4 方言失败、3 超时，不能替代权威结果。原 P18 不擅自改类型，当前配置在模型请求前拦下不支持接口，正式比较必须单列此环境限制。证据见

validation/codesys_release_v1.1/ 、 validation/toolchain_smoke_v1.1/ 、 validation/matiec_live_r2/ 及 docs/codesys_validation.md。它们不能替代至少 5 题真实 Agent 日志、成员 4 功能验证与跨机复现。

第 4 章 实验结果与分析（统稿版）

数据截至 2026-11-01。以下沿用原版自研框架的正式实验，不纳入任何后续框架改进试跑。原始记录未修改。数字来自统计输出/逐题逐组结果.csv、34 题四组矩阵.csv 和 四组汇总.csv；每行可回溯到 result.json。

4.1 实验设计与数据合并

题集为 P01—P20、S01—S14，共 34 题。E1 为 deepseek-v4.1-flash + Aider 0.86.2；E2 为 gpt-6.1-sol + Aider；E3 为 deepseek-v4.1-flash + 原版自研框架；E4 为 gpt-6.1-sol + 原版自研框架。四组共用 CODESYS V3.5 SP22 Patch 4、20 ms 扫描周期及组员 4 的逐拍功能用例。种子 42 只作为本地运行标识，没有发送给 API，因此单次输出不保证逐字复现。

原首轮四组各 34 条记录。P01、P08、P13、P14、P17、P18、P19、S11 修订判据后，四组各补跑 8 题，共 32 条新记录。最终矩阵用这 32 条替换原首轮相应题目的旧记录，其余 26 题沿用首轮；旧记录保存供追溯，不与新记录相加。已核对最终逐题表 136 行、每组 34 行、矩阵 34 道不同题目。

4.2 指标口径

“整题通过”要求权威编译通过，且该题所有有效功能用例通过。34 题均纳入严格分母；预算终止、超时、工具错误、编译失败与功能测试失败按原状态列出。编译通过不等于功能通过。耗时采用每题端到端 latency_s；p95 由 34 个耗时样本线性插值。token 仅累计 API 返回的已知 usage，未知用量不记为零，金额须另核对平台账单及当时单价。

按实际执行的用例计，E1 为 155/197（78.68%），E2 为 147/203（72.41%），E3 为 72/175（41.14%），E4 为 94/209（44.98%）。这些分母分别来自 31、32、28、33 道进入功能测试的题目；未编译通过或未运行的题目没有测试用例计入分母。因此用例通过率受进入测试阶段的题目数量影响，不能替代 34 题统一分母的整题通过率，也不能单凭该比率对模型排序。

4.3 四组结果

表 4-1 四组核心结果

组别	模型	框架	整题通过	严格通过率	编译通过	耗时中位数	耗时 p95
E1	deepseek-v4.1-flash	Aider	18/34	52.94%	32/34	553.0 s	830.4 s
E2	gpt-6.1-sol	Aider	16/34	47.06%	32/34	299.7 s	594.7 s
E3	deepseek-v4.1-flash	原版自研	8/34	23.53%	28/34	357.1 s	540.5 s
E4	gpt-6.1-sol	原版自研	7/34	20.59%	33/34	268.8 s	636.4 s

按最终状态分解：E1 有 13 题功能未全过、2 题超时、1 题工具错误；E2 有 16 题功能未全过、1 题编译失败、1 题超时；E3 有 24 题预算终止（19 题有可编译产物、5 题无可接受产物）、2 题超时；E4 有 25 题功能未全过、1 题编译失败、1 题超时。超时和工具问题不能直接写成模型生成能力不足。

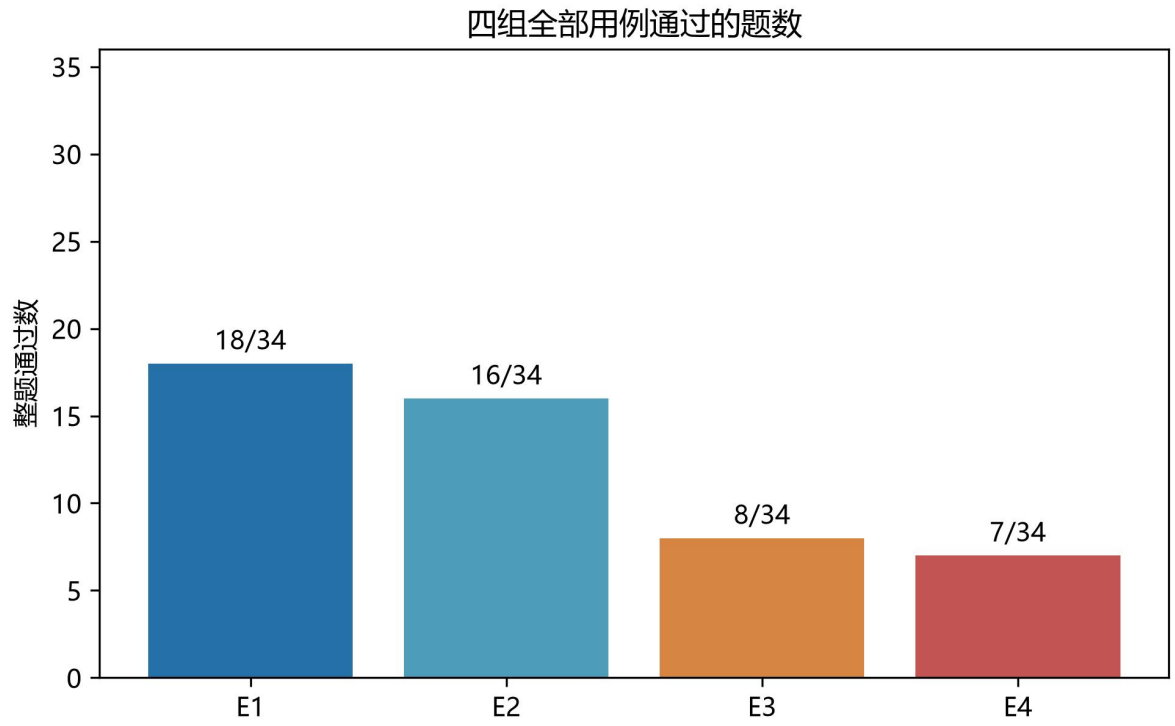


图 4-1 四组整题通过数

4.4 难度分层与配对比较

表 4-2 按难度分层的整题通过数

组别	简单题通过	中等题通过	困难题通过
E1	6/9	8/14	4/11
E2	7/9	6/14	3/11
E3	5/9	2/14	1/11
E4	4/9	2/14	1/11

同模型比较框架时，DeepSeek 的 E1/E3 共同通过 8 题，仅 E1 通过 10 题，仅 E3 通过 0 题；GPT 的 E2/E4 共同通过 7 题，仅 E2 通过 9 题，仅 E4 通过 0 题。同框架比较模型时，Aider 的 E1/E2 共同通过 15 题、仅 E1 通过 3 题、仅 E2 通过 1 题；自研的 E3/E4 共同通过 6 题、仅 E3 通过 2 题、仅 E4 通过 1 题。

这些数字说明在本题集、本机配置和本次运行中，Aider 组的严格整题通过数高于原版自研组。模型间差异只有 1—2 题，且两种模型的 token 用量、响应速度及框架停止方式不同；单次运行不足以估计随机波动，不能据此给模型作普遍排名。

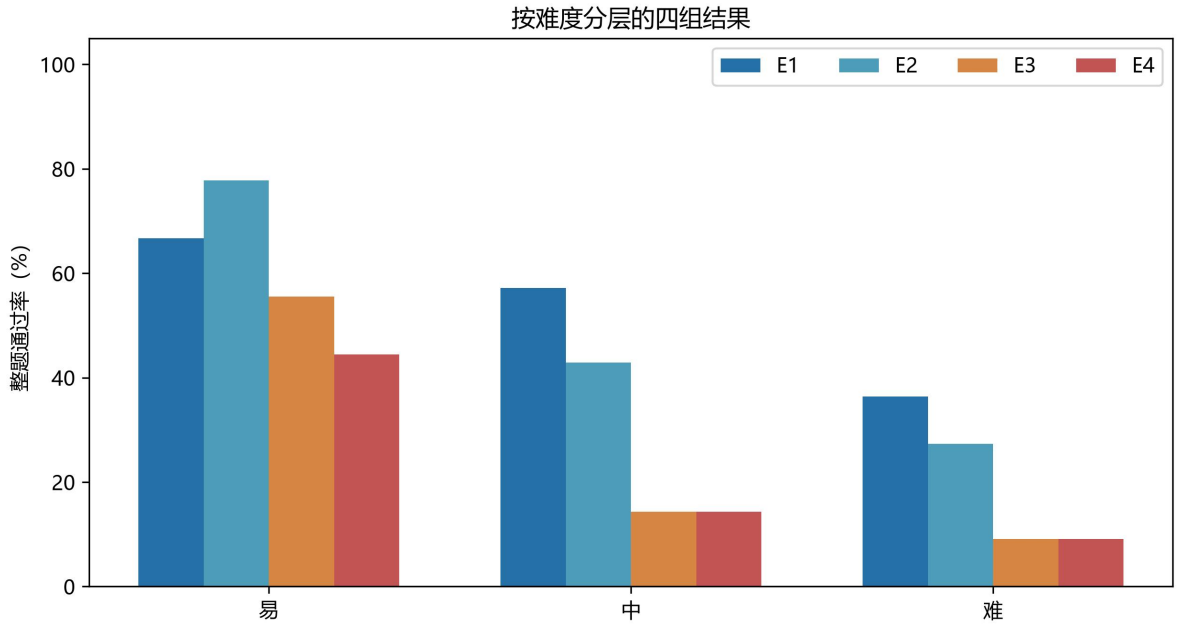


图 4-2 按难度分层的整题通过率

4.5 失败归因与判据限制

E3 的 24 次预算终止尤其需要单列。原版框架在累计预算检查中把请求和响应的 UTF-8 字节长度当成 token，与 100,000 token 门槛比较，可能过早停止修复。因此 E3 的低通过数同时受框架预算实现影响，不能全部归因于 DeepSeek 的代码能力。此处只解释原实验的局限，不更换原版框架，也不把后续试跑混入正式矩阵。

P19、S11 的题面与用例尚有冲突，具体位置见 判据待核查.md。S05-06 还存在输入 FlowPulse 连续两拍为 TRUE，却要求第二拍再次增加总量的预期，与“每个上升沿才计数”的题面相冲突。现有严格结果照原始记录报告，但这些争议题不能作为模型或框架优劣的决定性证据。若课程组统一判据，应冻结题面与用例版本，四组在相同条件下另建批次补测，保留旧记录。

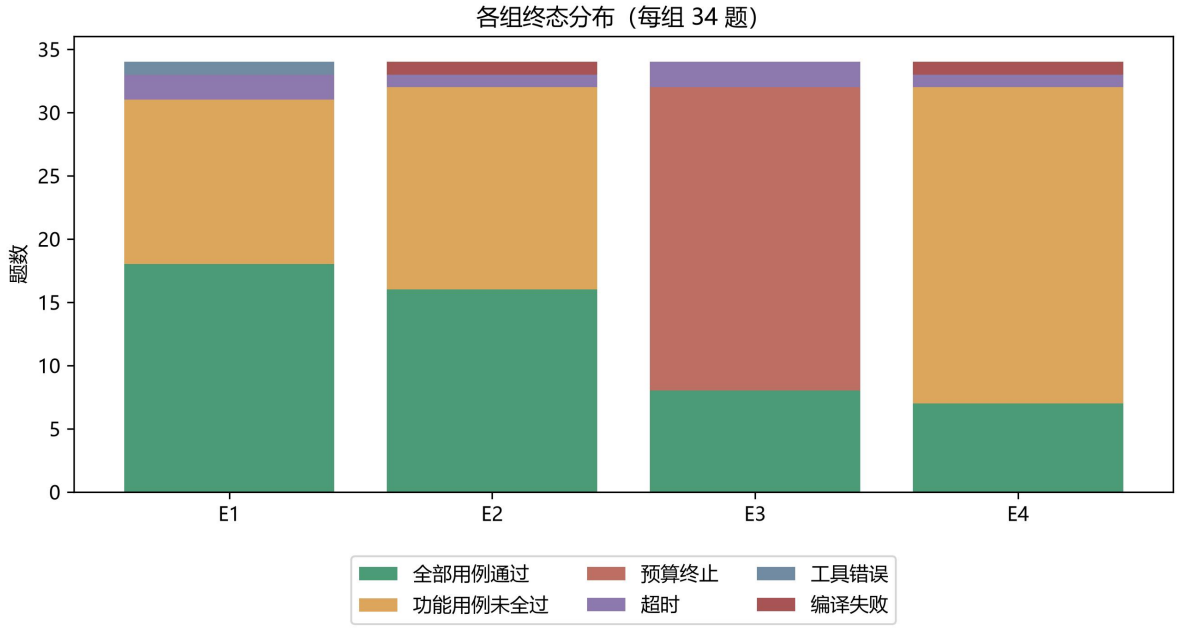


图 4-3 四组终态分布

4.6 资源消耗与可复现性

表中 token 为已知 usage 之和，E1 有 8 条、E2 有 1 条、E4 有 1 条用量未知；不能用已知总量直接比较全部费用。原版自研的 100,000 token 是本地停止门槛，不是平台实际收费上限。四组耗时包含模型请求、编译和逐拍测试；未统一为等 token 或等时间预算，横向结论必须保留这一限制。模型端未接收 seed，网关和编译环境也可能变化。

表 4-3 已知 token 用量

组别	已知输入 token	已知输出 token	用量未知记录
E1	537,229	987,897	8
E2	1,026,727	180,806	1
E3	103,954	943,390	0
E4	211,195	121,692	1

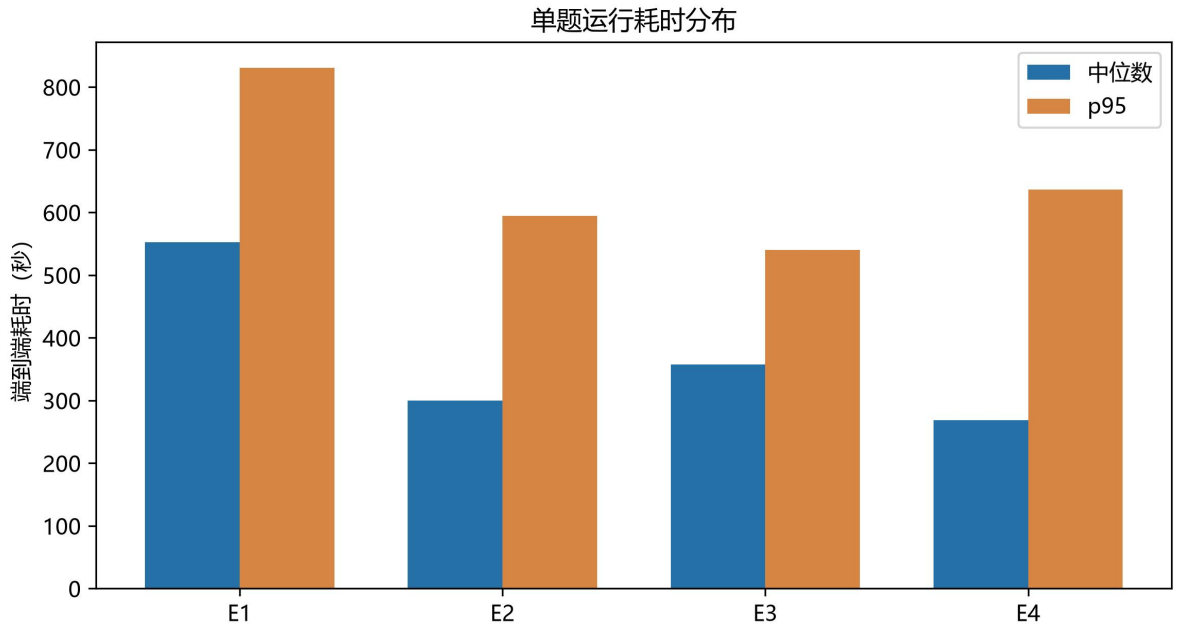


图 4-4 单题端到端耗时

4.7 交付与后续处理

交付物包括本章、四组汇总表、逐题逐组表、34 题矩阵、四张图、统计脚本、配置、原始 result.json 与编译/测试证据。组员 5 额外用仓库 ST 解释器独立重放了 132 个有源码 run 中的 72 个，60 个因解释器未支持的 ST 方言中止，4 个 abandoned run 没有源码；这些记录标记为 `independent_replay`，不冒充 CODESYS 原始 actual。五人分工清单另规划 12 题复跑；课程正文未将其列为四组主轮的硬性条件。若执行增强复跑，应独立标记，不覆盖本次 136 条最终矩阵。

结论与局限

本作业完成了一套可运行、可测试、可对比的 PLC ST 生成实验件。34 道题均具有结构化题面和 LTL 规约，Agent 能把生成、编译、逐周期测试和修复串成闭环，测试层能定位到具体周期和变量，四组实验能够用同一套脚本重算。这些产物使“代码是否编译”与“动作是否逐拍正确”成为两个独立、可检查的问题。

正式矩阵显示，在当前 34 题、同一实验机和本版配置下，Aider 搭配两个模型的严格整题通过数高于原版自研框架。这个结果有具体证据，但不能外推为 Aider 框架普遍更优。E3 的大量预算终止与自研框架的预算实现有关，P19、S11、S05-06 仍有判据冲突，模型随机波动也没有重复实验估计。更稳妥的结论是：编译通过高估了功能正确性；有界反馈对程序生成有价值；当前自研版本在预算管理和边界判据上仍需改进。

提交件的已知限制包括三点。第一，换机完整重跑尚未完成，环境表只能支持读者判断卡点，不能证明跨机一致。第二，精简实验包缺少完整逐拍 actual 中间文件，失败可回指但不可在包内重放。第三，部分题目的题面和用例仍有争议，相关失败不宜作为优劣结论的决定性证据。后续工作应优先修正预算计量、统一争议判据、保留完整 actual，再做多 seed 重复实验和跨机验证。

参考文献

[1] GenAI 赋能工业编程课程组. 期末大作业五人分工与交付清单: v3.0[Z]. 2026.

[2] International Electrotechnical Commission. IEC 61131-3: Programmable controllers - Part 3: Programming languages[S]. Geneva: IEC, 2013.

[3] NAMUR. NE 43: Signal level for fault information from measuring transducers[S]. Leverkusen: NAMUR, 2003.

[4] International Organization for Standardization. ISO 4413: Hydraulic fluid power - General rules and safety requirements for systems and their components[S]. Geneva: ISO, 2010.

[5] CODESYS GmbH. CODESYS Development System command line and scripting documentation[EB/OL]. https://content.helpme-codesys.com/.

[6] Beremiz Project. MATIEC compiler source and command-line documentation[EB/OL]. https://github.com/beremiz/matiec.

[7] Aider AI. Aider documentation: OpenAI-compatible APIs, scripting and lint/test integration[EB/OL]. https://aider.chat/docs/.

[8] Zertz777. Gen-AI: PLC ST generation and cycle-level verification repository[EB/OL]. https://github.com/zertz777/Gen-AI.